

**Subject:** CS336 Assignment 1, Data Loading and Batching

**Date:** from 2026/8/18 to 2026/8/19

---

## **Contents**

|                                  |          |
|----------------------------------|----------|
| <b>Data Loading and Batching</b> | <b>1</b> |
|----------------------------------|----------|

## Data Loading and Batching

### 训练任务的本质：下一个 Token 预测

语言模型的训练目标是：给定前  $m$  个词，预测第  $m + 1$  个词。为了高效训练，在一个长度为  $L$  的窗口内，我们不仅预测最后一个词，而是让模型在每一个位置都预测它的下一个词。

- 输入  $x$ . 从索引  $i$  开始、长度为  $L$  的序列： $[x_i, x_{i+1}, \dots, x_{i+L-1}]$ ;
- 目标  $y$ . 输入序列向后平移一位、长度同样为  $L$ ： $[x_{i+1}, x_{i+2}, \dots, x_{i+L}]$ .

**物理意义.** 当模型看到输入中的第一个词  $x_i$  时，它应该输出  $x_{i+1}$ ；看到  $[x_i, x_{i+1}]$  时，应输出  $x_{i+2}$ . 这种偏移设计允许我们在一个计算步内完成  $L$  次学习。

### 代码实现逻辑拆解

由于数据集通常极大（几百 GB），我们不能使用传统的 list，而是基于 Numpy 数组（通常配合内存映射 mmap 使用）进行操作。

**Step 1: 确定合法边界.**

```
max_idx = n - max_seq_length - 1
```

为什么要减 1？因为目标序列  $y$  需要取到  $i + L + 1$  的位置；如果不减 1，当随机抽到序列末尾时，取  $y$  会发生越界错误。

**Step 2: 随机采样起始点.**

```
ix = torch.randint(0, max_idx + 1, (batch_size,))
```

一次性生成 batch\_size 个随机数，作为本批次中每个句子的“起点”。这种随机性保证模型每一轮看到的组合都不一样，有利于泛化。

**Step 3: 内存切片与堆叠.**

```
x_stack = [dataset[i : i + max_seq_length] for i in ix]
y_stack = [dataset[i + 1 : i + max_seq_length + 1] for i in ix]
```

利用 Python 列表推导式从 Numpy 数组中提取片段，此时数据仍在 CPU 内存中。

**Step 4: 张量化与类型转换.**

```
x = torch.from_numpy(np.array(x_stack)).to(device).long()
```

原始数据通常是 uint16 或 int32，但 PyTorch 的 Embedding 层要求输入必须是 int64 (Long)，因此显式调用 .long() 是防止报错的关键。

### 优化：内存映射 (np.memmap)

这是大模型训练的标配技术。在 main\_train.py 中，不会直接 f.read() 整个文件，而是使用：

```
data = np.memmap('train.bin', dtype=np.uint16, mode='r')
```

- **原理.** memmap 并不真正把数据读入 RAM，而是在磁盘和逻辑内存之间建立映射；
- **配合 get\_batch.** 当 get\_batch 执行切片操作时，操作系统才会去磁盘上精确地把那几 KB 数据拉进缓存；
- **价值.** 这让你能在只有 16GB 内存的个人电脑上，轻松训练存储在 500GB 硬盘里的数据集。